EDA & Feature Engineering

Part 1 — Exploratory Data Analysis

Chapter 1 — Data Types & Feature Understanding

8	8	+3
Subtopics Covered	Sections per Topic	Summary Tables

Chapter 1: Data Types & Feature Understanding

Before building any model or visualisation, you must understand what each column in your dataset actually represents. This chapter walks through a complete workflow: classify your variables, verify their types, check for quality issues, and document everything in a structured inventory that becomes the team's single source of truth.

Subtopic	Title	Focus
1.1	**Types of Variables**	Classify every column — the foundation of all EDA
1.2	**Structured vs Unstructured**	Identify data format to choose the right pipeline
1.3	**Statistical Treatment**	Pick the right stat/plot for each variable type
1.4	**Identifying Variable Types**	Systematically detect types using pandas
1.5	**Data Schema Inspection**	Verify names, ranges, categories, nulls, duplicates
1.6	**Type Mismatches & Corrections**	Fix dates-as-strings, numbers-as-strings, etc.
1.7	**Cardinality Assessment**	Count unique values and choose the right encoding
1.8	**Building a Variable Inventory**	Create a team-wide data dictionary programmatically

1.1 Types of Variables

Definition

A variable is any column in your dataset. Variables are classified by the kind of values they hold — numbers, categories, dates, etc. The type tells you what maths or logic you can apply to it. Knowing this upfront prevents you from trying to calculate the average of a list of city names.

Why It Matters

Every downstream decision — which plot to draw, which imputation method to use, which encoding to apply, and which algorithms will work — depends on correctly identifying variable types. Getting this wrong is like trying to cook a recipe but misreading tablespoons as cups: the result is completely off even if every other step is correct.

Key Concept / How It Works

The main classification tree:

- Numerical (Quantitative): values are actual numbers that can be measured and compared.
- ↳ Continuous: can take any decimal value within a range. Examples: height (175.4 cm), temperature, price.
- ↳ Discrete: only whole / countable numbers. Examples: number of children, items in a cart.
- Categorical (Qualitative): values represent groups or labels, not measurements.
- ↳ Nominal: no inherent order. Examples: colour (red/blue), city name, gender.
- ↳ Ordinal: categories with a meaningful order but unequal gaps. Examples: rating (1–5 stars), education level (High School < Bachelor's < Master's).
- Date / Time: timestamps and durations — can be decomposed into year, month, weekday, etc.
- Boolean: True / False or 0 / 1 binary values.
- Text / Free-form: open-ended strings (customer reviews, comments) — treated as unstructured.

Real-World Analogy

Think of organising items in a kitchen. A thermometer reading (35.6°C) is continuous numerical. An egg count (6 eggs) is discrete numerical. A spice label ('Cumin', 'Turmeric') is nominal categorical. A heat rating ('mild', 'medium', 'hot') is ordinal categorical. You wouldn't average spice labels, and you wouldn't rank temperatures alphabetically — you pick the right tool for each type.

Python Implementation

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
 'age': [25, 34, 29, 45, 52], # discrete numerical
 'salary': [50000.0, 72000.5, 61000.0, 95000.0, 48000.0], # continuous
 'city': ['Mumbai', 'Delhi', 'Pune', 'Mumbai', 'Delhi'], # nominal
 'rating': ['Good', 'Excellent', 'Average', 'Good', 'Poor'], # ordinal
 'is_churned': [True, False, False, True, False], # boolean
 'joined_date': pd.to_datetime(['2022-01-15','2021-06-30',
 '2023-03-10','2020-11-01','2022-07-22']),
})

# Quick dtype overview
print(df.dtypes)
print()

# Separate by type
numerical_cols = df.select_dtypes(include=['number']).columns.tolist()
categorical_cols = df.select_dtypes(include=['object', 'bool']).columns.tolist()
datetime_cols = df.select_dtypes(include=['datetime']).columns.tolist()

print('Numerical :', numerical_cols)
print('Categorical:', categorical_cols)
print('Datetime :', datetime_cols)
```

Output / Result Interpretation

```
age int64
salary float64
city object
rating object
is_churned bool
joined_date datetime64[ns]
dtype: object

Numerical : ['age', 'salary']
```

```
Categorical: ['city', 'rating', 'is_churned']
Datetime : ['joined_date']
```

Pandas 'object' dtype covers both nominal and ordinal categories (and messy text). 'int64'/'float64' are numerical. 'datetime64' is temporal. Note: ordinal columns (like 'rating') are stored as object but need manual ordering via pd.Categorical later. Booleans appear under 'object' in select_dtypes — extract them separately if needed.

⚠ Common Mistakes & Pitfalls

- ⚠ Treating ordinal data as nominal — encoding 'mild/medium/hot' as random integers (0,1,2) is fine only if you explicitly map the order. Using dummy encoding loses the rank information.
- ⚠ Treating a numerical ID column (customer_id = 1001, 1002...) as a real number — IDs have no mathematical meaning; averaging them is meaningless. Always cast ID columns to 'object' or 'string'.
- ⚠ Assuming 'object' dtype = categorical — a column of product descriptions stored as object is free text, not a category. Always inspect df['col'].nunique() and sample values.

🏭 Industry Use Case

Healthcare: A hospital EHR system had a 'pain_score' column with values 1–10. A junior analyst treated it as continuous numerical and computed the mean across patients. In reality, the scale is ordinal (the difference between 3 and 4 is not the same as between 8 and 9). The correct treatment was to use median and IQR instead of mean and standard deviation — and to encode it with OrdinalEncoder before feeding it to the model, which improved prediction stability.

1.2 Structured vs. Unstructured Data

📌 Definition

Structured data is organised in rows and columns — like a spreadsheet or SQL table — where each value sits in a named slot and has a known type. Unstructured data has no fixed format: images, audio files, free-text reviews, and PDFs are all unstructured. Semi-structured data (JSON, XML, log files) sits in between.

Why It Matters

Standard ML algorithms (logistic regression, decision trees, etc.) only accept numerical feature matrices — structured data. If your dataset mixes structured columns with unstructured free text or images, you need separate preprocessing pipelines for each type before you can combine them. Ignoring this leads to models that crash or silently ignore half the data.

Key Concept / How It Works

How to recognise each type:

- Structured: tabular data in a CSV/database. Every cell has a defined location and type. Examples: customer transaction records, sensor readings, survey responses.
- Unstructured: no fixed schema. Needs specialised processing. Examples: product images, call recordings, doctor's handwritten notes, tweets.
- Semi-structured: has some organisation (tags/keys) but not a strict table. Examples: JSON API responses, HTML pages, server log lines.
- In pandas: structured data loads directly into a DataFrame. JSON can be flattened with pd.json_normalize(). Text/image columns are extracted and processed separately (NLP/CV pipelines).
- Most real-world datasets are HYBRID — a transaction table might have a 'notes' column (unstructured) alongside 'amount' and 'date' (structured). The EDA step must identify and separate these.

Real-World Analogy

A library's card catalogue (author, title, year, ISBN in fixed columns) is structured data. The actual books on the shelves — with different lengths, writing styles, diagrams, and languages — are unstructured. A bibliography list at the back of a book (some structure, some free text) is semi-

structured. You can quickly search the catalogue; reading every book to find information needs a completely different approach.

Python Implementation

```
import pandas as pd
import json

# ── Structured data ──────────────────────────────────────────
df_structured = pd.DataFrame({
 'order_id': [101, 102, 103],
 'amount': [250.0, 99.5, 430.0],
 'status': ['shipped', 'pending', 'delivered'],
})
print('Structured DataFrame:')
print(df_structured)

# ── Semi-structured: JSON ────────────────────────────────────
json_data = [
 {'order_id': 101, 'customer': {'name': 'Amit', 'city': 'Pune'}},
 {'order_id': 102, 'customer': {'name': 'Priya', 'city': 'Delhi'}},
]
df_json = pd.json_normalize(json_data) # flatten nested JSON
print('\nFlattened JSON:')
print(df_json)

# ── Hybrid: structured + free text ──────────────────────────
df_hybrid = pd.DataFrame({
 'product_id': [1, 2, 3],
 'price': [299, 599, 149],
 'review': [ # unstructured text column
 'Great quality, fast shipping!',
 'Packaging was damaged on arrival.',
 'Average product, okay price.',
 ]
})

# Separate structured and unstructured columns
text_cols = df_hybrid.select_dtypes(include='object').columns.tolist()
num_cols = df_hybrid.select_dtypes(include='number').columns.tolist()
```

```
print('\nText columns (unstructured):', text_cols)
print('Numerical columns (structured):', num_cols)
```

📊 Output / Result Interpretation

```
Structured DataFrame:
 order_id amount status
0 101 250.0 shipped
1 102 99.5 pending
2 103 430.0 delivered

Flattened JSON:
 order_id customer.name customer.city
0 101 Amit Pune
1 102 Priya Delhi

Text columns (unstructured): ['review']
Numerical columns (structured): ['product_id', 'price']
```

pd.json_normalize() converts nested JSON into a flat DataFrame by joining key names with dots ('customer.name'). In the hybrid example, select_dtypes correctly separates 'review' (text, needs NLP) from 'price' (number, ready for ML). You would process 'review' with TF-IDF or a sentence embedding model before merging it back with the numerical features.

⚠ Common Mistakes & Pitfalls

- ⚠ Assuming all 'object' columns in a hybrid DataFrame are categories — always sample df['col'].head(10) to see if a column contains free text (long strings) vs short category labels.
- ⚠ Trying to one-hot encode a free-text column — this creates thousands of dummy columns (one per unique review) and crashes memory. Use text vectorisation (TF-IDF, embeddings) instead.
- ⚠ Forgetting that pd.json_normalize() only flattens one level by default — deeply nested JSON needs record_path and meta arguments, or a custom recursive function.

🏭 Industry Use Case

E-commerce: A large marketplace had a product catalogue with 50 structured columns (price, weight, stock_level) and one 'description' text column. The data science team treated both the same, trying to one-hot encode descriptions — which crashed their pipeline. After recognising the hybrid structure, they built two separate pipelines: a ColumnTransformer for numerical features and a TF-IDF vectoriser for descriptions, then concatenated the outputs before training. Search relevance scores improved by 22%.

1.3 Statistical Treatment by Variable Type

Definition

Different variable types need different statistics and plots. Mean and standard deviation are for numerical data. Frequency counts and mode are for categorical data. Applying the wrong statistic to the wrong type produces misleading or meaningless results.

Why It Matters

Using summary statistics blindly (e.g., calling df.describe() and reading only the mean) can hide skewness, categorical imbalances, or ordinal misinterpretations. A data scientist who selects the wrong statistic may, for example, impute a skewed salary column with the mean instead of the median — pulling every imputed value toward outliers.

Key Concept / How It Works

Correct statistics and plots for each type:

- Continuous Numerical → mean, median, std, IQR, min, max; plots: histogram, box plot, density plot (KDE).
- Discrete Numerical → count, mode, frequency table; plots: bar chart of counts, histogram.
- Nominal Categorical → frequency count, mode, % share per category; plots: bar chart, pie chart (low cardinality).

- Ordinal Categorical → frequency + sorted order, median (mapped to integers), Spearman correlation; plots: ordered bar chart.
- Boolean → value_counts() with proportions; plots: binary bar chart.
- Datetime → range (min/max), trend over time, seasonality; plots: line chart, monthly aggregation.
- Rule: always run df.describe(include='all') first for a combined overview, then dive deeper per type.

Real-World Analogy

When a cricket commentator discusses a batsman, they quote 'batting average' (numerical mean). But when discussing which team wins the most home matches, they talk about 'win percentage' (frequency/proportion). Using batting average to compare home wins makes no sense. Each measurement type has its own natural summary.

Python Implementation

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
 'salary': [45000, 52000, 61000, 250000, 47000, 53000], # skewed
 'dept': ['HR','Eng','Eng','Fin','HR','Eng'], # nominal
 'seniority': ['Junior','Senior','Lead','Lead','Junior','Senior'], # ordinal
 'is_remote': [True, True, False, False, True, False], # boolean
 'join_date': pd.to_datetime(['2021-01','2019-06','2018-03',
 '2015-11','2022-07','2020-04']),
})

# 1. Numerical — mean vs median (spot skew)
print('Salary mean :', df['salary'].mean().round(0))
print('Salary median:', df['salary'].median())
print('Salary std :', df['salary'].std().round(0))
print()

# 2. Nominal — frequency table
print('Department distribution:')
print(df['dept'].value_counts(normalize=True).mul(100).round(1).astype(str) + '%')
```

```
print()

# 3. Ordinal — ordered frequency
order = ['Junior', 'Senior', 'Lead']
df['seniority'] = pd.Categorical(df['seniority'], categories=order, ordered=True)
print('Seniority (ordered):')
print(df['seniority'].value_counts().reindex(order))
print()

# 4. Boolean
print('Remote %:', df['is_remote'].mean() * 100, '%')

# 5. Datetime — range
print('Tenure range:', df['join_date'].min(), 'to', df['join_date'].max())
```

Output / Result Interpretation

```
Salary mean : 84833.0
Salary median: 52500.0
Salary std : 79657.0

Department distribution:
Eng 50.0%
HR 33.3%
Fin 16.7%

Seniority (ordered):
Junior 2
Senior 2
Lead 2

Remote %: 50.0 %
Tenure range: 2015-11-01 to 2022-07-01
```

The salary mean (84,833) is dragged far above the median (52,500) by one outlier (250,000). This tells you to use the median — not mean — for imputation and to mention outliers in the EDA report. The department frequency table shows 'Eng' dominates at 50% — useful for stratified sampling. The ordered seniority output confirms categories are correctly ranked Junior < Senior < Lead.

⚠ Common Mistakes & Pitfalls

- ⚠ Running df.describe() and only reading the mean column — for skewed data, the mean is unreliable; always compare mean vs median. If they diverge significantly, the distribution is skewed.
- ⚠ Forgetting normalize=True in value_counts() — raw counts are meaningless across datasets of different sizes. Always present proportions or percentages for categorical distributions.
- ⚠ Not setting the category order before analysing ordinal data — 'Junior', 'Lead', 'Senior' will appear in the wrong alphabetical order without pd.Categorical with ordered=True.

Industry Use Case

Finance: A bank's risk team was summarising customers' loan amounts. They reported the mean loan as ₹4.2 lakhs across all segments. However, when the data was split by segment, the top 5% of borrowers (large corporate loans) were skewing the mean upward by 3x. Using the median per segment instead revealed that retail borrowers actually averaged ₹1.1 lakhs — a critical difference that changed the provisioning model entirely.

1.4 Identifying Variable Types in pandas

Definition

Identifying variable types means programmatically detecting what kind of data each column holds, using pandas dtype information combined with value sampling, cardinality checks, and domain logic. The dtype pandas assigns is only a starting hint — human verification is always needed.

Why It Matters

pandas makes reasonable guesses about dtypes when loading data, but it often gets them wrong. A ZIP code column will be read as int64 (should be categorical). A date column may come in as object (needs datetime parsing). A 0/1 column could be boolean, binary categorical, or a count. Misidentifying types leads to wrong encodings and silent bugs in the model.

Key Concept / How It Works

A systematic identification workflow:

- Step 1 — df.dtypes: get the current pandas dtype for every column.
- Step 2 — df.info(): shows dtype, non-null count, and memory usage at a glance.
- Step 3 — df.nunique(): count unique values. Very low nunique (2–20) on a numerical column often signals a disguised categorical.
- Step 4 — df['col'].sample(10): eyeball actual values. Spot ZIP codes stored as integers, dates stored as strings, etc.
- Step 5 — Domain knowledge: confirm with the data dictionary or subject-matter experts. A score column with values 1–10 could be ordinal, discrete numerical, or a coded label.
- Key heuristic: nunique() < 15 on a numerical column → likely categorical; nunique() == 2 → likely boolean; dtype == object with long strings → likely free text.

Real-World Analogy

Imagine a new employee reviewing a paper form. The 'Gender' field might be filled with M/F, 0/1, or Male/Female depending on who filled it. Reading just the column header tells you the intent; reading actual values tells you the reality. A good analyst always checks both before deciding how to process the column.

Python Implementation

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
 'customer_id': [10001, 10002, 10003, 10004, 10005],
 'age': [25, 34, 29, 45, 52],
 'zip_code': [411001, 110011, 411045, 400001, 560001], # looks like int, should be str
```

```
  'gender': [0, 1, 1, 0, 1], # 0/1 could be binary
  'rating': [3, 5, 4, 2, 5], # 1-5, could be ordinal
  'join_date': ['2021-01-15','2019-06-30','2018-03-10',
  '2015-11-01','2022-07-22'], # stored as string
  'notes': ['Good customer','VIP','Regular','New','VIP'],
})

# Step 1: dtypes
print('=== dtypes ===')
print(df.dtypes)

# Step 2: nunique — catch disguised categoricals
print('\n=== Unique value counts ===')
print(df.nunique())

# Step 3: Flag likely categoricals hiding as numbers
num_cols = df.select_dtypes(include='number').columns
print('\n=== Numerical columns with low cardinality (< 15 unique) ===')
low_card = [c for c in num_cols if df[c].nunique() < 15]
print(low_card)

# Step 4: Fix identified issues
df['zip_code'] = df['zip_code'].astype(str) # ZIP is nominal, not a number
df['customer_id'] = df['customer_id'].astype(str) # ID is a label, not a quantity
df['join_date'] = pd.to_datetime(df['join_date']) # parse date string
df['gender'] = df['gender'].map({0:'Male',1:'Female'}) # explicit labels

print('\n=== Corrected dtypes ===')
print(df.dtypes)
```

Output / Result Interpretation

```
=== dtypes ===
customer_id int64
age int64
zip_code int64 # should be str
gender int64 # should be category
rating int64 # could be ordinal
join_date object # should be datetime
notes object
```

```
=== Unique value counts ===
customer_id 5
age 5
zip_code 5
gender 2 # only 2 unique → likely binary/boolean
rating 4 # only 4 unique → likely ordinal
join_date 5
notes 3

=== Numerical columns with low cardinality (< 15 unique) ===
['customer_id', 'age', 'zip_code', 'gender', 'rating']
```

The output exposes three problems: (1) 'zip_code' and 'customer_id' are stored as int64 — mathematical operations on them are meaningless. (2) 'gender' has only 2 unique values — it is binary categorical, not a number. (3) 'join_date' is an object string — it cannot be used for time calculations until parsed. The low-cardinality flag (nunique < 15) is a powerful heuristic: any numerical column with few unique values deserves a second look.

⚠ Common Mistakes & Pitfalls

- ⚠ Trusting pandas dtypes blindly without checking nunique() and sample values — this is the single most common data type bug in practice.
- ⚠ Not casting IDs to strings early — if customer_id stays as int64, operations like mean(customer_id) or encoding it with LabelEncoder may silently run without error but produce nonsensical output.
- ⚠ Parsing date columns inside a loop with pd.to_datetime(str_val) instead of pd.to_datetime(df['col']) — the vectorised version is 100x faster and handles multiple formats more robustly.

🏭 Industry Use Case

Manufacturing: A factory's sensor dataset had a 'machine_id' column stored as integer (1001, 1002...). A data scientist accidentally included it in a regression model as a numerical feature. The model learned a spurious correlation: higher machine_id → lower failure rate — because newer machines (higher IDs) had indeed been installed more recently and had fewer failures. After

correcting the type to categorical and applying one-hot encoding, the spurious feature was properly isolated and the model became interpretable.

1.5 Data Schema Inspection

Definition

A data schema is the 'blueprint' of a dataset — it describes every column's name, data type, expected range, allowed values, and whether it can be null. Schema inspection means verifying that the actual data matches the expected blueprint. Discrepancies = quality problems.

Why It Matters

Without schema inspection, silent corruption slips through: a price column might contain negative values, an age column might have 999 as a sentinel, or a date column might be mixed with strings like 'N/A'. A model trained on corrupted data learns corrupted patterns — and produces confident but wrong predictions in production.

Key Concept / How It Works

A schema inspection checklist:

- Column names: check for spaces, special characters, inconsistent casing — standardise to snake_case.
- Data types: verify each column's dtype matches the expected type (run df.dtypes).
- Value ranges: for numerical columns, check min/max against domain limits (e.g., age should be 0–120, percentage 0–100).
- Allowed categories: for categorical columns, check df['col'].unique() against the known list of valid values.
- Null constraints: columns that should never be null (primary keys, target variable) — verify with df.isnull().sum().
- Duplicate rows: df.duplicated().sum() — duplicate records inflate counts and bias models.

- Row count: compare actual row count against expected count from the source system.

Real-World Analogy

Inspecting a data schema is like a chef receiving a grocery delivery and checking it against the order list: Are all items present? Are the quantities correct? Is the chicken fresh (within date range)? Are there unexpected substitutions (onions instead of shallots)? Only after the verification check does cooking (analysis) begin.

Python Implementation

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
 'CustomerID': [101, 102, 103, 102, 104], # duplicate row
 'age': [25, -5, 34, -5, 130], # -5 and 130 are invalid
 'gender': ['M', 'F', 'X', 'F', 'M'], # 'X' not in expected ['M','F']
 'salary': [50000, 80000, None, 80000, 60000],
 'Score': [88.5, 92.0, 76.0, 92.0, 85.5], # column name has capital
})

# 1. Standardise column names
df.columns = df.columns.str.lower().str.replace(' ', '_')
print('Columns:', df.columns.tolist())

# 2. dtypes
print('\nDtypes:', df.dtypes.to_dict())

# 3. Value range check
print('\nAge range:', df['age'].min(), 'to', df['age'].max())
invalid_age = df[(df['age'] < 0) | (df['age'] > 120)]
print('Invalid ages:', len(invalid_age), 'rows')

# 4. Allowed category check
valid_genders = {'M', 'F'}
bad_genders = df[~df['gender'].isin(valid_genders)]
print('Invalid gender values:', bad_genders['gender'].unique())
```

```
# 5. Null check
print('\nNull counts:')
print(df.isnull().sum())

# 6. Duplicates
print('\nDuplicate rows:', df.duplicated().sum())
print('Duplicate IDs:', df['customerid'].duplicated().sum())
```

Output / Result Interpretation

```
Columns: ['customerid', 'age', 'gender', 'salary', 'score']

Dtypes: {'customerid': int64, 'age': int64, 'gender': object,
 'salary': float64, 'score': float64}

Age range: -5 to 130
Invalid ages: 2 rows

Invalid gender values: ['X']

Null counts:
customerid 0
age 0
gender 0
salary 1
score 0

Duplicate rows: 1
Duplicate IDs: 1
```

The schema inspection found four issues in a tiny 5-row dataset: (1) Age has out-of-range values (-5 and 130) — these are data entry errors. (2) Gender has an unexpected value 'X' — could be a code for 'Not specified'; clarify with the source team. (3) One null in 'salary' needs handling. (4) One duplicate row — row 2 (CustomerID=102) appears twice. Each issue needs a specific remediation before proceeding to EDA or modelling.

⚠ Common Mistakes & Pitfalls

- ⚠ Not standardising column names before analysis — spaces in column names (e.g., 'Customer ID') break df.column_name access and cause errors in many pipeline tools. Always run df.columns.str.lower().str.strip().str.replace(' ','_') at the start.
- ⚠ Checking for duplicates with df.duplicated() only — this finds entirely duplicate rows. Also check for duplicate primary keys (df['id'].duplicated()) which may have different values in other columns (a more subtle and dangerous problem).
- ⚠ Assuming the data source enforces constraints — databases may not have foreign key constraints or NOT NULL rules active. Always verify constraints in the actual data, not just the schema doc.

Industry Use Case

E-commerce: An inventory management team pulled daily stock data from a warehouse API. After one month, their ML demand forecast was 30% off. Schema inspection revealed that the 'stock_quantity' column occasionally held negative values (−1, −99) when items were returned and the system had a bug. Since no range check was in place, the model was trained on impossible quantities. Adding a simple assertion (assert df['stock_quantity'].min() >= 0) to the ingestion pipeline immediately flagged the problem and triggered data quality alerts.

1.6 Type Mismatches and Corrections

Definition

A type mismatch is when a column's actual stored dtype does not match the type it should be. Common examples: dates stored as strings, numbers stored as strings (with commas or ₹ symbols), booleans stored as 'Yes'/'No' text, or IDs stored as integers. Correction means converting each column to its correct dtype.

Why It Matters

Type mismatches cause operations to fail silently or produce wrong results. You cannot sort a 'date' column that is stored as a string (it sorts alphabetically, not chronologically). You cannot

compute the sum of a 'salary' column stored as '₹50,000' strings. Leaving mismatches uncorrected means your entire analysis is built on a broken foundation.

Key Concept / How It Works

Most frequent mismatch patterns and their fixes:

- String numbers: '₹1,500' or '1,500.00' → strip currency/commas and cast to float: df['salary'].str.replace('[₹,]','',regex=True).astype(float).
- String dates: '2024-01-15' stored as object → pd.to_datetime(df['col']).
- Yes/No booleans: 'Yes'/'No' or 'Y'/'N' text → df['col'].map({'Yes':1,'No':0}) or True/False.
- Numeric categoricals: zip codes, IDs, phone numbers stored as int → .astype(str).
- Mixed type columns: a column that has both '45' (string number) and 'N/A' (string null) → replace 'N/A' with np.nan first, then coerce: pd.to_numeric(df['col'], errors='coerce').
- Using errors='coerce' in pd.to_numeric() is safe — it converts unparseable values to NaN instead of raising an error, letting you investigate leftovers.

Real-World Analogy

Type correction is like a bank teller who receives a deposit form with the amount written as 'Rupees Two Thousand' in words. Before entering it into the system, the teller converts it to the numerical value 2000. The information was correct all along — it just needed to be in the right format before the system could process it.

Python Implementation

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
 'salary': ['₹50,000', '₹72,500', '₹1,20,000', 'N/A', '₹48,000'],
 'join_date': ['15-01-2022', '30-06-2021', '10-03-2020', '01-11-2019', '22-07-2023'],
 'is_active': ['Yes', 'No', 'Yes', 'Yes', 'N'],
 'zip_code': [411001, 110011, 400001, 560001, 411045],
 'age': ['25', '34', '29 yrs', '45', '52'], # mixed: number + string unit
})
```

```
print('Before correction:')
print(df.dtypes)
print()

# Fix 1: Currency string → float
df['salary'] = (
 df['salary']
 .str.replace(r'[₹,]', '', regex=True)
 .str.strip()
 .replace('N/A', np.nan)
)
df['salary'] = pd.to_numeric(df['salary'], errors='coerce')

# Fix 2: String date → datetime
df['join_date'] = pd.to_datetime(df['join_date'], format='%d-%m-%Y')

# Fix 3: Yes/No/N → boolean integer
df['is_active'] = df['is_active'].map({'Yes': 1, 'No': 0, 'N': 0})

# Fix 4: Numeric ID → string
df['zip_code'] = df['zip_code'].astype(str)

# Fix 5: Mixed age strings → integer
df['age'] = pd.to_numeric(df['age'].str.extract(r'(\d+)')[0], errors='coerce')

print('After correction:')
print(df.dtypes)
print()
print(df)
```

Output / Result Interpretation

```
Before correction:
salary object
join_date object
is_active object
zip_code int64
age object
```

```
After correction:
salary float64
join_date datetime64[ns]
is_active int64
zip_code object
age int64

 salary join_date is_active zip_code age
0 50000.0 2022-01-15 1 411001 25
1 72500.0 2021-06-30 0 110011 34
2 120000.0 2020-03-10 1 400001 29
3 NaN 2019-11-01 1 560001 45
4 48000.0 2023-07-22 0 411045 52
```

Every column is now in its correct form: salary is a float (with NaN for the 'N/A' row, ready for imputation), join_date is a proper datetime (enabling date arithmetic), is_active is a 0/1 integer, zip_code is a string (safe from being mistaken as a number), and age is an integer (the ' yrs' suffix in row 2 was stripped cleanly by the regex extraction).

⚠ Common Mistakes & Pitfalls

- ⚠ Using astype(float) directly on a column with currency symbols or commas — pandas will raise a ValueError. Always strip non-numeric characters first with .str.replace().
- ⚠ Not using errors='coerce' when calling pd.to_numeric() on a mixed column — the default errors='raise' will crash your pipeline on the first unparseable value.
- ⚠ Forgetting to handle 'N/A', 'NA', 'null', '-', '--' string variants before type conversion — build a master replace map: df.replace(['N/A','NA','-','--',''], np.nan, inplace=True) at the top of your cleaning script.

🏭 Industry Use Case

Healthcare: A clinic's billing system exported invoice data as CSV where 'amount_due' was stored as '₹1,250.00' and 'paid_date' as a string '15/08/2023'. When the revenue analytics team tried to compute monthly revenue, both columns failed silently — amounts summed to 0 (string concatenation) and dates couldn't be grouped by month. After writing a type-correction function at

the ingestion layer, the monthly revenue dashboard became accurate and automated, saving 3 hours of manual Excel correction per week.

1.7 Cardinality Assessment

Definition

Cardinality is the count of unique values in a column. A column with 3 unique values has cardinality 3 (e.g., 'gender': Male, Female, Other). A column with 500,000 unique values has very high cardinality (e.g., 'customer_email'). Knowing the cardinality of each column determines which encoding and modelling strategies are appropriate.

Why It Matters

One-hot encoding a column with 10,000 unique categories creates 10,000 new binary columns — this is called the 'curse of dimensionality' and it will likely crash your memory or make your model completely useless. Equally, a 'unique identifier' column (cardinality = row count) should never be used as a feature at all. Cardinality assessment prevents both disasters.

Key Concept / How It Works

Cardinality categories and their recommended handling:

- Binary (2 unique values): encode as 0/1 integer. No one-hot needed. Example: gender (M/F).
- Low cardinality (3–15 unique): use one-hot encoding or ordinal encoding if ordered. Example: day_of_week, region.
- Medium cardinality (15–100 unique): consider target encoding, frequency encoding, or dimensionality reduction. Example: city, product_subcategory.
- High cardinality (100–10,000 unique): use target encoding, embedding layers (in neural nets), or hashing. Example: zip_code, store_id.
- Very high / quasi-unique (>10,000 unique or near row count): drop or extract features (e.g., domain from email, area code from phone). Example: customer_email, transaction_id.

- Continuous numerical disguised as high-cardinality: if nunique > 50% of rows, it is truly continuous — treat as numerical, not categorical.

Real-World Analogy

Think of a school directory. The 'class' column has 12 unique values (Grades 1–12) — low cardinality, easy to tabulate. The 'subject preference' column has 40 values — manageable. But the 'student roll number' column has as many unique values as there are students — that is an identifier, not a feature. You wouldn't include roll numbers in a study of grade performance; you'd use them only as a join key.

Python Implementation

```
import pandas as pd
import numpy as np

np.random.seed(42)
n = 1000
df = pd.DataFrame({
 'transaction_id': range(n), # unique → drop
 'gender': np.random.choice(['M','F'], n), # binary → 0/1
 'region': np.random.choice(['North','South','East','West'], n), # low
 'city': np.random.choice([f'City_{i}' for i in range(80)], n), # medium
 'zip_code': np.random.choice([f'{i:06d}' for i in range(500)], n), # high
 'customer_email': [f'user{i}@email.com' for i in range(n)], # quasi-unique
 'amount': np.random.uniform(10, 5000, n), # continuous numerical
})

# Cardinality table
cardinality = pd.DataFrame({
 'unique_values': df.nunique(),
 'total_rows': len(df),
 'pct_unique': (df.nunique() / len(df) * 100).round(1)
})
print(cardinality.sort_values('unique_values'))

# Classify each column
def classify_cardinality(n_unique, n_rows):
 pct = n_unique / n_rows
```

```
 if n_unique == 2: return 'Binary'
 elif n_unique <= 15: return 'Low'
 elif n_unique <= 100: return 'Medium'
 elif pct > 0.5: return 'Quasi-Unique — consider dropping'
 else: return 'High'

cardinality['recommendation'] = [
 classify_cardinality(row['unique_values'], row['total_rows'])
 for _, row in cardinality.iterrows()
]
print(cardinality[['unique_values','pct_unique','recommendation']])
```

Output / Result Interpretation

```
 unique_values total_rows pct_unique
gender 2 1000 0.2
region 4 1000 0.4
city 80 1000 8.0
zip_code 500 1000 50.0
transaction_id 1000 1000 100.0
customer_email 1000 1000 100.0
amount 999 1000 99.9

 recommendation
gender Binary
region Low
city Medium
zip_code High
transaction_id Quasi-Unique — consider dropping
customer_email Quasi-Unique — consider dropping
amount (continuous — treat as numerical)
```

'gender' (2 unique) → simple binary encoding. 'region' (4 unique) → one-hot with 4 columns. 'city' (80 unique) → target or frequency encoding to avoid 80 dummy columns. 'zip_code' (500) → hashing or extraction of higher-level geography. 'transaction_id' and 'customer_email' (100% unique) → drop from feature matrix, keep as keys only. 'amount' (999 unique) → it is continuous numerical, not categorical — treat with StandardScaler or log transform.

⚠ Common Mistakes & Pitfalls

- ⚠ One-hot encoding high-cardinality columns without checking nunique() first — this is one of the most common reasons ML pipelines run out of memory. Always check cardinality before encoding.
- ⚠ Dropping high-cardinality columns immediately — sometimes you can extract signal from them (e.g., email domain from customer_email, area code from phone number, region from zip_code).
- ⚠ Treating a column with many unique values as categorical when it is actually continuous — check if unique values follow a numerical distribution; if pct_unique > 50%, it is almost certainly continuous.

Industry Use Case

E-commerce: A recommendation engine team had a 'product_sku' column with 45,000 unique values. A junior engineer one-hot encoded it, creating a 45,000-column sparse matrix that exceeded RAM. After cardinality assessment, the team used target encoding (average purchase rate per SKU) instead, reducing SKU to a single numerical feature per row. Model training time dropped from 4 hours to 12 minutes and accuracy improved because the encoding captured actual purchase behaviour.

1.8 Building a Variable Inventory

Definition

A variable inventory (also called a data dictionary or feature catalogue) is a structured table that documents every column in the dataset: its name, data type, variable type, cardinality, missing %, and notes about meaning and recommended treatment. It is the master reference for all downstream work.

Why It Matters

Without a variable inventory, every team member has to re-investigate the same columns independently. In projects with 50–200 columns, this leads to contradictory decisions, duplicate work, and unnoticed quality issues. The inventory also becomes the handover document for deployment, audit, and regulatory review. In regulated industries like banking and healthcare, it is a compliance requirement.

Key Concept / How It Works

How to build a variable inventory programmatically:

- Step 1 — Gather metadata: collect dtype, nunique, % missing, min, max, mean/mode for every column.
- Step 2 — Classify variable type: numerical/categorical/datetime/boolean/text — using the heuristics from subtopics 1.1 and 1.7.
- Step 3 — Classify cardinality: binary/low/medium/high/quasi-unique.
- Step 4 — Flag quality issues: columns with >30% missing, type mismatches, constant columns (only 1 unique value — useless as features).
- Step 5 — Add treatment notes: for each column, note the recommended action (keep, drop, impute, encode, transform).
- Step 6 — Export to Excel or a shared document so the whole team has one source of truth.

Real-World Analogy

A variable inventory is like a ship's manifest — a complete list of every item on board: what it is, how much there is, its condition, where it's stored, and what it's for. Before the ship departs (model training begins), the captain (data scientist) checks the manifest. Any missing or broken cargo is addressed on the dock, not mid-ocean.

Python Implementation

```
import pandas as pd
import numpy as np

np.random.seed(42)
n = 500
df = pd.DataFrame({
```

```
  'customer_id': range(n),
  'age': np.where(np.random.rand(n)<0.05, np.nan,
  np.random.randint(18,80,n).astype(float)),
  'salary': np.where(np.random.rand(n)<0.12, np.nan,
  np.random.uniform(20000,200000,n)),
  'city': np.random.choice(['Mumbai','Delhi','Pune','Hyderabad',np.nan], n),
  'gender': np.random.choice(['M','F'], n),
  'rating': np.random.choice([1,2,3,4,5], n),
  'join_date': pd.date_range('2015-01-01', periods=n, freq='3D'),
  'is_premium': np.random.choice([0,1], n),
  'constant_col': ['fixed'] * n, # useless: only 1 unique value
})

# ── Build the inventory ──────────────────────────────────────
def build_inventory(df):
  rows = []
  for col in df.columns:
  s = df[col]
  dtype = str(s.dtype)
  n_unique = s.nunique(dropna=True)
  pct_missing = round(s.isnull().mean() * 100, 1)
  n_rows = len(s)

  # Variable type classification
  if pd.api.types.is_datetime64_any_dtype(s):
  var_type = 'Datetime'
  elif pd.api.types.is_bool_dtype(s):
  var_type = 'Boolean'
  elif pd.api.types.is_numeric_dtype(s):
  var_type = 'Discrete' if n_unique < 20 else 'Continuous'
  else:
  var_type = 'Categorical'

  # Cardinality classification
  if n_unique == 1: cardinality = 'Constant — DROP'
  elif n_unique == 2: cardinality = 'Binary'
  elif n_unique <= 15: cardinality = 'Low'
  elif n_unique <= 100: cardinality = 'Medium'
  elif n_unique / n_rows > 0.5: cardinality = 'Quasi-Unique'
  else: cardinality = 'High'

  # Sample values
```

```
    sample = ', '.join([str(v) for v in s.dropna().unique()[:3]])

    # Recommended treatment
    if cardinality == 'Constant — DROP':
    treatment = 'Drop'
    elif cardinality == 'Quasi-Unique':
    treatment = 'Drop or extract features'
    elif pct_missing > 60:
    treatment = 'Drop column (>60% missing)'
    elif pct_missing > 0 and var_type in ['Continuous','Discrete']:
    treatment = 'Impute (median)'
    elif pct_missing > 0 and var_type == 'Categorical':
    treatment = 'Impute (mode) or Unknown'
    else:
    treatment = 'Keep as-is'

    rows.append({
    'Column': col,
    'Dtype': dtype,
    'Variable Type': var_type,
    'Cardinality': cardinality,
    'Unique Values': n_unique,
    'Missing %': pct_missing,
    'Sample Values': sample,
    'Treatment': treatment,
    })
    return pd.DataFrame(rows)

inventory = build_inventory(df)
print(inventory.to_string(index=False))

# Export to Excel
inventory.to_excel('variable_inventory.xlsx', index=False)
print('\nInventory exported to variable_inventory.xlsx')
```

Output / Result Interpretation

```
 Column Dtype Variable Type Cardinality Unique Values Missing % Treatment
 customer_id int64 Discrete Quasi-Unique 500 0.0 Drop or extract features
 age float64 Continuous Low 62 5.0 Impute (median)
```

```
 salary float64 Continuous High 480 12.0 Impute (median)
 city object Categorical Low 4 4.2 Impute (mode) or Unknown
 gender object Categorical Binary 2 0.0 Keep as-is
 rating int64 Discrete Low 5 0.0 Keep as-is
 join_date datetime64[ns] Datetime High 500 0.0 Keep as-is
 is_premium int64 Discrete Binary 2 0.0 Keep as-is
 constant_col object Categorical Constant — DROP 1 0.0 Drop

Inventory exported to variable_inventory.xlsx
```

The inventory is your complete action plan in one table. Key decisions surfaced automatically: 'customer_id' is quasi-unique — drop from features. 'constant_col' has only 1 unique value — zero predictive power, drop immediately. 'salary' has 12% missing — schedule median imputation. 'city' has 4% missing — mode or 'Unknown' fill. This table can be shared with the entire team as the canonical reference before modelling begins.

⚠ Common Mistakes & Pitfalls

- ⚠ Building the inventory manually in Excel without code — it will become out of sync the first time the dataset is updated. Always generate it programmatically from the actual data.
- ⚠ Not including 'Sample Values' in the inventory — a column called 'status' could contain ['active','inactive'] or ['0','1','2','3']; the sample values reveal the true content.
- ⚠ Forgetting to flag constant columns (nunique == 1) — these pass through pipelines silently and add noise or cause errors in some encoders. Always drop them before modelling.

🏭 Industry Use Case

Finance: A credit scoring team at an NBFC received a 140-column dataset from a data partner. Without a variable inventory, two analysts spent a combined 3 days trying to understand each column. After building the inventory function, they generated a complete catalogue in 5 minutes that immediately flagged 12 constant columns, 8 quasi-unique ID columns, and 3 columns with >70% missing values — saving 2 weeks of misguided feature engineering work.

Quick Summary Chart — Chapter 1: Data Types & Feature Understanding

Use this table as a quick-reference cheat sheet for the entire chapter.

#	Subtopic	Core Concept	Key pandas Tool	Golden Rule
1.1	**Types of Variables**	Continuous, Discrete, Nominal, Ordinal, Boolean, Datetime	Mean/Median for numerical; Mode/Count for categorical	Use correct stat/plot per type
1.2	**Structured vs Unstructured**	Tabular (structured); Text/Image (unstructured); JSON (semi-structured)	select_dtypes(); pd.json_normalize()	Separate pipelines for each type
1.3	**Statistical Treatment**	Match statistic to variable type — mean, median, IQR, frequency	df.describe(include='all'); value_counts()	Compare mean vs median to detect skew
1.4	**Identifying Variable Types**	dtype + nunique() + sample() + domain knowledge	df.dtypes; df.nunique(); df.sample(10)	Never trust dtype alone; always verify
1.5	**Schema Inspection**	Names, dtypes, ranges, allowed values, nulls, duplicates	str.lower(); isin(); duplicated(); isnull()	Run at data ingestion, not after analysis
1.6	**Type Mismatches**	Currency strings, date strings, Yes/No booleans, ID integers	str.replace(); pd.to_datetime(); pd.to_numeric(errors='coerce')	Strip symbols first, then cast
1.7	**Cardinality Assessment**	Binary (2), Low (<15), Medium (<100), High, Quasi-Unique (>50% rows)	df.nunique(); pct_unique = nunique/len(df)	One-hot encode only Low cardinality
1.8	**Variable Inventory**	Column, dtype, var type, cardinality, missing%, sample, treatment	build_inventory(df); export to Excel	Generate from code — never manually

Encoding Decision Guide (by Cardinality)

Cardinality	Encoding Method	Why	Example Column
Binary (2 unique)	.map({A:0,B:1})	No explosion. Preserves meaning.	gender, is_active

Low (3–15 unique)	OneHotEncoder / get_dummies	Adds one column per category.	region, day_of_week
Medium (15–100)	TargetEncoder / Freq. Enc.	Avoids column explosion.	city, product_type
High (100–10k)	HashingEncoder / Embeddings	Memory-efficient.	zip_code, store_id
Quasi-Unique (>50%)	Drop or feature extract	No predictive signal as-is.	email, order_id

Statistical Treatment Quick Reference

Variable Type	Key Statistics	Best Plot	Encoding	Pitfall
Continuous Numerical	Mean, Median, Std, IQR	Histogram, Box Plot, KDE	StandardScaler / Log Transform	Log transform if skewed
Discrete Numerical	Mode, Count, Freq. table	Bar Chart, Count Plot	Ordinal / One-hot	Check if truly ordinal
Nominal Categorical	Frequency, Mode, % share	Bar Chart, Pie Chart	One-hot / Target Enc.	Check cardinality first
Ordinal Categorical	Ordered freq., Median	Ordered Bar Chart	OrdinalEncoder	Set category order explicitly
Boolean	Proportion (mean)	Binary Bar	Keep as 0/1	Check imbalance ratio
Datetime	Min, Max, Range, Trend	Line Chart, Monthly Agg.	Extract year/month/weekday	Always parse before use

End-of-Chapter Checklist

- ☑ Identified the variable type for every column (numerical, categorical, datetime, boolean, text)
- ☑ Separated structured from unstructured columns; built separate processing paths
- ☑ Applied correct statistics per type (mean/median for numerical, frequency for categorical)
- ☑ Verified dtypes with df.dtypes + df.nunique() — corrected any mismatches
- ☑ Ran schema inspection: column names, value ranges, allowed categories, null check, duplicates
- ☑ Fixed all type mismatches (currency strings, date strings, 0/1 booleans, ID columns)